

Claude Code

A brownbag workshop at



by Seth House

@whiteinge
seth@eseth.com

Onboarding

Start with `CLAUDE.md`

Write down the particulars of your codebase:

Start with `CLAUDE.md`

Write down the particulars of your codebase:

- Noteworthy file locations.
- How to run the linter, formatter, and test suite.
- Architectural principles and code conventions.
- Commit expectations.
- Idiosyncrasies and known temporary placeholders.

Source of truth

Code and version-controlled files are authoritative. Sessions, prompts, and memory are layers on top.

CLAUDE .md hierarchy

CLAUDE.md hierarchy

- `~/.claude/CLAUDE.md` — your preferences across all projects.

CLAUDE.md hierarchy

- `~/.claude/CLAUDE.md` — your preferences across all projects.
- `<project>/CLAUDE.md` — rules for this codebase.

CLAUDE.md hierarchy

- `~/.claude/CLAUDE.md` — your preferences across all projects.
- `<project>/CLAUDE.md` — rules for this codebase.
- `<project>/weirdModule/CLAUDE.md` — rules just for that subdirectory.

Path-specific rules — `.claude/rules/`

Markdown files with glob frontmatter; load on-demand when Claude reads a matching path.

```
---
```

```
paths: ["src/auth/**/*"]
```

```
---
```

Auth code uses `bcrypt` for password hashing – never plaintext.
Session tokens always flow through `SessionManager`.

Define success criteria

“Before showing the result: run the linter, run the formatter, and run the test suite.”

Reference other files with @

```
@.claude/api-docs.md  
@docs/architecture.md
```

Always be updating **CLAUDE .md**

Always be updating `CLAUDE.md`

- After a session goes sideways, ask Claude what was missing.
- Remove redundancies and conflicting bits.
- Add the rule that would have prevented the confusion.

When Claude is wrong, fix the code first

When Claude is wrong, fix the code first

- A clearer docstring or function name often beats another `CLAUDE.md` rule.
- Define a “golden master” file for Claude to pattern-match against.
- Migrate legacy code to the new pattern only when modifying it, in a separate “update old patterns” commit.

Whitelist commands deliberately

The permission system is configurable, not fuzzy:

Whitelist commands deliberately

The permission system is configurable, not fuzzy:

- `default` — prompts for risky actions.
- `acceptEdits` — auto-allows file writes.
- `plan` — read-only mode.
- `bypassPermissions` — allows everything (be careful).

Working with Claude

Use the CLI, not the web

Even for one-off, “I just need to know X” questions.

Plan mode

A read-only mode. Claude can research and reason, but can't change files until you approve a plan.

`Shift+Tab` to cycle, or `/plan` to jump in.

Tell Claude to propose plan mode

In `CLAUDE.md`:

“For architectural changes or work touching multiple modules, propose plan mode before starting.”

Tell Claude to bail early

“If an approach fails or gets complicated, STOP and re-plan — don’t keep pushing down a broken path.”

Don't accept “done” without proof

“Never mark a task complete without proving it works (run tests and demonstrate correctness).”

Ask Claude what it needs

It's surprisingly good at telling you what's ambiguous.

Ask Claude to push back

AI agents will happily do what you ask — which can be a problem when you've made a mistake.

But you can also ask them to push back and point out any problems in your thinking.

Ask Claude (seriously, ask)

Don't just tell. Consult.

Ask Claude (seriously, ask)

Don't just tell. Consult.

- “What do you think of this approach?”
- “What would you add to this design?”
- “What are the trade-offs between X and Y?”

Or: instruct, then leave room

“Do X — but point out if you see a better way.”

Commit early, rebase later

Make incremental `wip` commits between Claude operations.

For amendments to earlier commits: `git commit --fixup=<sha>` — no message required.

File format quirks

Markdown

Long-line-per-idea formatting works better than hard-wrapped prose.

JSON

Prefer line-per-entry formatting over fully minified or fully pretty-printed.

Subagents

Subagents start fresh

Subagents start fresh

- They don't inherit your conversation or your prompt;
- they *do* read `CLAUDE.md`.
- Brief them like a colleague who just walked into the room.

When to reach for subagents

For sequential, conversational work — design, architecture review, debugging — a single thread with focused prompts usually wins.

Reach for subagents when:

When to reach for subagents

For sequential, conversational work — design, architecture review, debugging — a single thread with focused prompts usually wins.

Reach for subagents when:

- Multiple lookups can run in parallel and don't depend on each other.

When to reach for subagents

For sequential, conversational work — design, architecture review, debugging — a single thread with focused prompts usually wins.

Reach for subagents when:

- Multiple lookups can run in parallel and don't depend on each other.
- You want noisy or large output kept out of the main session.

When to reach for subagents

For sequential, conversational work — design, architecture review, debugging — a single thread with focused prompts usually wins.

Reach for subagents when:

- Multiple lookups can run in parallel and don't depend on each other.
- You want noisy or large output kept out of the main session.
- You need a wide-search-space exploration with isolated context.

Useful slash commands

`/resume` — find an old session

Lists previous sessions; fuzzy-find supported.

`/btw` — ask a side question

Doesn't add to the conversation history.

/status — token usage and cost

Check current model, version, and session name.

Session

Total cost: \$35.04

Total duration (API): 1h 10m 42s

Total duration (wall): 22h 26m 36s

Total code changes: 2350 lines added, 257 lines removed

Usage by model:

claude-haiku-4-5: 5.4k input, 4.8k output, 233.4k cache read, 1

claude-opus-4-7: 408 input, 291.9k output, 39.7m cache read, 1

Current session



18% used

Resets 8:30pm (America/Denver)

Current week (all models)



9% used

Resets Apr 30, 11pm (America/Denver)

`/remote-control` — work from your phone

Makes the session available via claude.ai.

Make your own slash commands

Make your own slash commands

- `~/.claude/commands/<name>.md` — personal, across all projects.

Make your own slash commands

- `~/.claude/commands/<name>.md` — personal, across all projects.
- `<project>/.claude/commands/<name>.md` — shared with the team via git.

Example: a `/wrap-up` command

End-of-session housekeeping:

Example: a `/wrap-up` command

End-of-session housekeeping:

- Review memory and `CLAUDE.md` for stale or duplicate entries.

Example: a `/wrap-up` command

End-of-session housekeeping:

- Review memory and `CLAUDE.md` for stale or duplicate entries.
- Propose migrating personal memory to `CLAUDE.md` when it's become a team convention.

Example: a `/wrap-up` command

End-of-session housekeeping:

- Review memory and `CLAUDE.md` for stale or duplicate entries.
- Propose migrating personal memory to `CLAUDE.md` when it's become a team convention.
- Write a session summary to `WIP.md` so the next session can pick up cleanly.

Esc once to interrupt

Single Esc stops the current operation.

Input shortcuts

Input shortcuts

- `Ctrl+J` — newline without submitting (multi-line input).

Input shortcuts

- `Ctrl+J` — newline without submitting (multi-line input).
- `Ctrl+G` or `Ctrl+X Ctrl+E` — open current input in `$EDITOR`.

Input shortcuts

- `Ctrl+J` — newline without submitting (multi-line input).
- `Ctrl+G` or `Ctrl+X Ctrl+E` — open current input in `$EDITOR`.
- `Ctrl+R` — search history.

Input shortcuts

- `Ctrl+J` — newline without submitting (multi-line input).
- `Ctrl+G` or `Ctrl+X Ctrl+E` — open current input in `$EDITOR`.
- `Ctrl+R` — search history.
- `Ctrl+V` — paste an image.

Input shortcuts

- `Ctrl+J` — newline without submitting (multi-line input).
- `Ctrl+G` or `Ctrl+X Ctrl+E` — open current input in `$EDITOR`.
- `Ctrl+R` — search history.
- `Ctrl+V` — paste an image.
- `Ctrl+L` — redraw; clear input.

Input shortcuts

- `Ctrl+J` — newline without submitting (multi-line input).
- `Ctrl+G` or `Ctrl+X Ctrl+E` — open current input in `$EDITOR`.
- `Ctrl+R` — search history.
- `Ctrl+V` — paste an image.
- `Ctrl+L` — redraw; clear input.
- Emacs-style `Ctrl+A/E/W` work if your terminal sets them.

Input shortcuts

- `Ctrl+J` — newline without submitting (multi-line input).
- `Ctrl+G` or `Ctrl+X Ctrl+E` — open current input in `$EDITOR`.
- `Ctrl+R` — search history.
- `Ctrl+V` — paste an image.
- `Ctrl+L` — redraw; clear input.
- Emacs-style `Ctrl+A/E/W` work if your terminal sets them.
- `/keybindings` — customize your own.
- `/config` — enable Vim mode!

Watch the CLI output

Status lines and command transitions surface tips you'd otherwise miss.

Epilogue

This deck was drafted by Claude Code from a messy, simple, first-draft outline.

Epilogue

This deck was drafted by Claude Code from a messy, simple, first-draft outline.

- It spotted factual mistakes in my outline and fixed them.

Epilogue

This deck was drafted by Claude Code from a messy, simple, first-draft outline.

- It spotted factual mistakes in my outline and fixed them.
- It restructured the flow into themed sections.

Epilogue

This deck was drafted by Claude Code from a messy, simple, first-draft outline.

- It spotted factual mistakes in my outline and fixed them.
- It restructured the flow into themed sections.
- It verified details (slash commands, `CLAUDE.md` hierarchy) via web lookups.

Epilogue

This deck was drafted by Claude Code from a messy, simple, first-draft outline.

- It spotted factual mistakes in my outline and fixed them.
- It restructured the flow into themed sections.
- It verified details (slash commands, `CLAUDE.md` hierarchy) via web lookups.
- It used the same `<x-inc>` and `<aside>` features this deck shows off.

Epilogue

This deck was drafted by Claude Code from a messy, simple, first-draft outline.

- It spotted factual mistakes in my outline and fixed them.
- It restructured the flow into themed sections.
- It verified details (slash commands, `CLAUDE.md` hierarchy) via web lookups.
- It used the same `<x-inc>` and `<aside>` features this deck shows off.

Eat the dog food.